

Smart Property Rating System

Full Project Description & Feature Specification

Version 1.0 | April 2026

1. Executive Summary

The Smart Property Rating System is a web-based tool that analyses any residential property or land parcel using multiple real-world factors and produces an easy-to-understand score out of 10. Rather than overwhelming buyers with raw data, the system synthesises distances, amenities, safety indicators, and price fairness into a single verdict with a plain-language explanation — telling buyers not just what the score is, but why.

Core value proposition: Same property, different score for different buyers — because a retired couple and a young professional have different priorities. The system adapts to the user, not the other way around.

The product is designed to grow in three phases: starting as a lightweight rating tool requiring no listings database, evolving into a community-powered platform, and ultimately becoming a full property marketplace differentiated by its built-in scoring layer.

2. Problem Statement

Property buyers in India — and globally — face a common frustration: portals like MagicBricks and 99acres show photos, prices, and locations, but provide no objective quality signal. Buyers must manually research transport links, look up crime data, estimate nearby school quality, and compare prices against area averages. This research takes days and is often unreliable.

The result is decision paralysis, uninformed purchases, and a trust gap between buyers and the market. First-time buyers and renters are most affected — they lack the local knowledge that experienced buyers accumulate over time.

Key pain points addressed	
No quality signal	Listings show features but never tell buyers if a property is actually good value
Research overload	Buyers must check 5+ sources to evaluate a single property
One-size scoring	Generic ratings ignore what matters to each buyer type

Safety opacity	Crime and safety data is scattered, stale, or inaccessible
Price confusion	No easy way to know if a price is fair for the area

3. Product Overview

The system collects structured data about a property — either entered manually by the user, or (in later phases) pulled automatically via APIs — and runs it through a weighted scoring algorithm. The output is a breakdown across four categories and an overall score, accompanied by a human-readable verdict.

Four scoring categories	
Connectivity	Distance to bus stops, railway stations, and airport. Weighted by mode importance.
Amenities	Proximity and count of hospitals, schools, and markets within a 2km radius.
Safety	Triangulated from official crime index, infrastructure checklist, community perception, and news recency.
Price fairness	Listing price vs area average, plus resale growth potential.

Each raw input is normalised to a 0–10 scale before combining. This ensures that a listing price of ₹55,00,000 and a bus distance of 0.4km are weighted on the same footing. Category scores are then combined using buyer-persona-specific weights to produce the final score.

4. Buyer Persona System

The most important design insight in this system is that the same property should score differently for different buyers. A property 5km from the nearest school is irrelevant to a bachelor but critical to a parent. The persona system solves this by adjusting category weights silently based on a short onboarding questionnaire.

Users answer 3 questions at onboarding — who they are buying for, whether they have school-age children, and whether this is for living or investment. The system assigns a persona automatically. Power users can override weights manually.

Buyer persona	Top priorities	Weight distribution
Family with kids	Schools, Safety	Amenities 35%, Safety 35%, Connectivity 20%, Price 10%
Young professional	Commute, Price	Connectivity 40%, Price 20%, Safety 20%, Amenities 20%

Retiree	Hospitals, Quiet	Amenities 40%, Safety 35%, Connectivity 15%, Price 10%
Investor	Resale, Price	Price Fairness 40%, Connectivity 25%, Amenities 20%, Safety 15%
Student / renter	Budget, Transport	Connectivity 40%, Price 30%, Safety 15%, Amenities 15%

Over time, as users rate whether recommendations were helpful, the system can use that feedback to fine-tune persona weights with real data — creating a learning loop that becomes a competitive moat.

5. Safety Scoring — Multi-Source Triangulation

Safety is the hardest category to score reliably. Official crime data in India (NCRB) lags reality by 1–2 years and is inconsistently reported across states. The system addresses this by combining four independent signals, each capturing a different dimension of safety.

Safety signal sources	
Official crime index	25% weight — NCRB or state police data. Low weight because data is often stale.
Infrastructure checklist	30% weight — Street lighting, CCTV coverage, security guards, gated entry. Verifiable and fresh.
Community perception	30% weight — User-submitted ratings, Google Maps review sentiment, local forum data.
News / incident recency	15% weight — How recently was an incident reported nearby? Uses local news APIs.

Transparency principle: Wherever safety data is older than 12 months or sourced from only one signal, the UI displays a warning: "Safety score based on limited data — verify locally." Admitting uncertainty builds more trust than hiding it.

For the early version (Phase 1), a structured safety survey replaces API data. The property lister or a field agent answers 8–10 yes/no questions: Are street lights functional? Is there a security guard? Are there active shops within 200m at night? This gives structured, fresh, verifiable data without any government dependency — and is already more reliable than most property portals in India.

6. Scoring Algorithm

6.1 Normalisation

Raw real-world values are normalised to 0–10 before weighting. Different functions are used depending on the nature of the input:

Normalisation functions	
normalize_distance(km)	Closer = better. Ideal and worst thresholds set per transport mode.
normalize_count(n)	More = better. Capped at an ideal count (e.g., 5 markets within 2km).
normalize_crime(index)	Inverted: low crime index = high score. $(100 - \text{index}) / 10$.
normalize_price_ratio	At or below area average = 10. More than 30% above average = 0.

6.2 Default weights

Default weights apply when no persona is selected or for anonymous users:

Default category weights	
Connectivity	30%
Amenities	30%
Safety	25%
Price fairness	15%

6.3 Verdict generation

After scoring, a rule-based verdict engine reads category scores and generates a plain-language summary. For example: "Good option overall. Strengths: excellent amenities, safe neighbourhood. Watch out for: overpriced for the area." The verdict is constructed from modular strength and warning clauses — no AI or ML required at this stage.

7. Technical Architecture

The system is built as a standard web application with a Python backend and a React frontend. The architecture is intentionally simple in Phase 1 to enable fast shipping, with clear extension points for data integrations and ML in later phases.

Technology stack

Backend	Python (Flask or FastAPI), PostgreSQL for property storage, Redis for caching API responses
Frontend	React with Tailwind CSS. Single-page app with slider-based input and live score updates.
Scoring engine	Pure Python — dataclasses for input validation, normalisation functions, weighted averages.
Data sources (Phase 2)	Google Maps Distance Matrix API, Google Places API, local crime open datasets, news APIs.
Auth	Simple email/Google OAuth for saved comparisons and history. Anonymous use supported.
Hosting	Any cloud provider (AWS, GCP, Railway.app). Docker containerised for easy deployment.

The API exposes a single core endpoint: POST /rate. It accepts a JSON payload with all property fields and returns a full scored result including category breakdowns, weights used, and the verdict. This makes it trivial to connect any frontend — web, mobile, or third-party integration.

8. Full Feature List

8.1 Phase 1 — Rating Tool (launch)

- Property score calculator with sliders for manual data entry
- Four-category breakdown: connectivity, amenities, safety, price fairness
- Overall score out of 10 with animated score ring
- Plain-language verdict with strength and warning tags
- Buyer persona selector — 5 personas with auto-adjusted weights
- Manual weight override for advanced users
- Safety checklist (8–10 yes/no questions replacing API data)
- Safety data transparency flag when data is limited or stale
- Responsive web UI — works on mobile and desktop

8.2 Phase 2 — Community Layer

- Auto-store every scored property in a database (anonymised)
- Locality score — rate a neighbourhood independent of any specific property
- Saved comparisons — score up to 5 properties and view side-by-side
- User-submitted safety ratings and reviews
- Google Maps API integration for automated distance calculation
- Google Places API for automated amenity count within radius
- "What's dragging my score down" breakdown panel
- Search by locality: "How does Koregaon Park score vs Baner?"

- User accounts with saved properties and history

8.3 Phase 3 — Full Platform

- Direct property listings from agents and sellers
- Every listing automatically pre-scored — score displayed on listing card
- Agent portal with listing management and analytics
- Promoted listings and verified badge system (monetisation)
- Lead generation — buyers can contact agents directly through the platform
- Score-based search filters: "Show only properties scoring above 7 on safety"
- ML-powered weight refinement based on user feedback and sale outcomes
- Mobile app (iOS and Android)

9. Product Roadmap

Phase	Name	Key Features	Cost
Phase 1 (0–6 months)	Rating Tool	Manual property input, scoring engine, verdict generator, persona selector, basic web UI	Near zero — just hosting
Phase 2 (6–18 months)	Community Layer	Auto-store scored properties, user reviews, safety surveys, locality scores, saved comparisons	Low — database + API costs
Phase 3 (18 months+)	Full Platform	Direct listings, agent portal, promoted listings, lead generation, verified badges	Medium — team + infrastructure

10. Monetisation Strategy

Individual buyers and renters are the primary users but have low willingness to pay directly. Monetisation is designed to come from the supply side — agents, developers, and financial services — while the tool remains free for buyers. This is the same model used by Zomato, Justdial, and most successful Indian consumer platforms.

Revenue streams by phase	
Phase 1	None — focus entirely on user acquisition and scoring accuracy validation.
Phase 2	Premium accounts for buyers (saved comparisons, history, PDF reports).
Phase 3	Promoted listings, verified agent badges, lead generation fees, developer dashboards.
Long-term	Home loan and insurance referral fees, data licensing to institutional investors.

11. Key Risks & Mitigations

Risks and mitigations	
Safety data quality	Multi-source triangulation + transparency flags. Never rely on a single government dataset.
Cold start (no data)	Phase 1 requires no database. Users bring their own properties. Data accumulates organically.
Scoring accuracy	Validate scores against actual sale prices. Collect user feedback on recommendation quality.
Competition from portals	MagicBricks/99acres have listings but no explainable scoring. That is the differentiation.
Low buyer monetisation	Free for buyers by design. Revenue comes from agents and developers in Phase 3.
Data freshness	Cache API responses with TTL. Flag scores older than 90 days for re-verification.

12. Immediate Next Steps

To launch Phase 1, the following items need to be completed in order:

- Finalise the normalisation thresholds for each input (requires local market research for Pune/target city).
- Build the safety checklist — 8 questions, yes/no format, tested with 10 real properties.
- Develop the React frontend with slider inputs and live score display.
- Deploy the Flask API on a cloud server with a public endpoint.
- Test with 20–30 real properties and compare scores against buyer intuition.
- Launch as a free tool, share in local housing WhatsApp groups and Reddit communities.
- Collect feedback on verdict accuracy and adjust weights accordingly.